

Aprendizaje por Refuerzo (o Reforzado) (Reinforcement learning (RL))

UMNG

2026

Agenda

- 1 Introducción
- 2 Aplicaciones Reales del Aprendizaje por Refuerzo
- 3 Historia y Evolución del RL
- 4 Componentes de un sistema de Aprendizaje por Refuerzo
- 5 Introducción Procesos de Decisión de Markov (MDP)
- 6 Características del Agente
- 7 Ecuaciones de Bellman

Aprendizaje humano es aprendizaje por refuerzo



Aprendizaje humano es aprendizaje por refuerzo



Aprendizaje humano es aprendizaje por refuerzo

Surgimiento de conductas de locomoción en entornos ricos



Aprendizaje humano es aprendizaje por refuerzo



Aprendizaje computacional



mapear situaciones a acciones

↙ **Aprendizaje + Refuerzo (recompensa)**

aprender a partir de la interacción

Definición: Aprendizaje por Refuerzo

“...es un enfoque computacional al aprendizaje a partir de la interacción, en el cual un agente aprende qué acciones tomar —cómo mapear situaciones a acciones— con el fin de maximizar una señal de recompensa acumulada, sin que se le diga explícitamente qué hacer.”

(adaptado de Reinforcement Learning: an introduction, Richard Sutton and Andrew Barto, 2015, 2nd Ed.)



Características clave

- **Mapeo de situaciones a acciones:** El agente aprende una *política* para decidir qué hacer en cada estado.
- **Maximización de recompensa:** Se optimiza una *señal numérica de refuerzo* acumulada en el tiempo.
- **Aprender por interacción:** El agente interactúa con un entorno sin una guía explícita.
- **Ciclo de lazo cerrado:** Las acciones afectan las futuras observaciones y recompensas.
- **Exploración vs. Explotación:** Balance entre probar nuevas acciones y usar lo ya aprendido.
- **Distinto de aprendizaje supervisado o no supervisado:** No se basa en datos etiquetados ni en estructura oculta.



Comparación de Paradigmas de Aprendizaje

Aprendizaje supervisado



el humano decide qué datos recolectar y cómo etiquetarlos

Aprendizaje no
supervisado



el humano decide qué datos recolectar

Aprendizaje por
refuerzo



*el **agente** determina de forma **autónoma** qué datos recolectar y cómo usarlos para su aprendizaje*

🎯 Juegos y Competencia Artificial

El Aprendizaje por Refuerzo ha marcado hitos históricos en juegos, demostrando su potencial desde entornos clásicos hasta competencias de nivel mundial.

- **1992 – TD-Gammon (Backgammon)** Gerald Tesauro (IBM) entrenó un agente que alcanzó nivel maestro usando $TD(\lambda)$ jugando contra sí mismo. [Vídeo](#)

🎯 Juegos y Competencia Artificial

El Aprendizaje por Refuerzo ha marcado hitos históricos en juegos, demostrando su potencial desde entornos clásicos hasta competencias de nivel mundial.

- **1992 – TD-Gammon (Backgammon)** Gerald Tesauro (IBM) entrenó un agente que alcanzó nivel maestro usando TD(λ) jugando contra sí mismo. [Vídeo](#)
- **2013–2014 – Pong (Atari)** DeepMind entrena una red neuronal con DQN para jugar Pong directamente desde píxeles. Marcó el inicio del Deep RL. [Video](#)

🎯 Juegos y Competencia Artificial

El Aprendizaje por Refuerzo ha marcado hitos históricos en juegos, demostrando su potencial desde entornos clásicos hasta competencias de nivel mundial.

- **1992 – TD-Gammon (Backgammon)** Gerald Tesauro (IBM) entrenó un agente que alcanzó nivel maestro usando TD(λ) jugando contra sí mismo. [Vídeo](#)
- **2013–2014 – Pong (Atari)** DeepMind entrena una red neuronal con DQN para jugar Pong directamente desde píxeles. Marcó el inicio del Deep RL. [Video](#)
- **2014–2015 – Breakout (Atari)** También con DQN, DeepMind entrena un agente que descubre estrategias inesperadas superando a humanos. [Breakout con Deep RL](#)

🎯 Juegos y Competencia Artificial

El Aprendizaje por Refuerzo ha marcado hitos históricos en juegos, demostrando su potencial desde entornos clásicos hasta competencias de nivel mundial.

- **1992 – TD-Gammon (Backgammon)** Gerald Tesauro (IBM) entrenó un agente que alcanzó nivel maestro usando TD(λ) jugando contra sí mismo. [Vídeo](#)
- **2013–2014 – Pong (Atari)** DeepMind entrena una red neuronal con DQN para jugar Pong directamente desde píxeles. Marcó el inicio del Deep RL. [Video](#)
- **2014–2015 – Breakout (Atari)** También con DQN, DeepMind entrena un agente que descubre estrategias inesperadas superando a humanos. [Breakout con Deep RL](#)
- **2016 – AlphaGo (Go)** DeepMind vence al campeón mundial de Go (Lee Sedol) con una combinación de RL profundo, redes neuronales y búsqueda Monte Carlo. [Sitio AlphaGo - DeepMind](#)



Robótica Autónoma – Locomoción por Refuerzo

Los robots pueden aprender a caminar y adaptarse a terrenos usando aprendizaje por refuerzo, sin programación explícita.

- **Spot subiendo escaleras:**

Stepping Up | Boston Dynamics

- **RealAnt en el mundo real:**

RealAnt robot con RL



Vehículos Autónomos – Conducción Inteligente

Los vehículos autónomos pueden aprender a conducir mediante prueba y error, optimizando maniobras seguras y eficientes.

- **Entrenamiento por refuerzo:**

Self-driving cars con RL

- **Wayve: Aprender a conducir en un día:**

Wayve - Learning to Drive



Finanzas – Optimización de Portafolios

El AR permite construir portafolios que se ajustan dinámicamente al mercado financiero, buscando maximizar ganancias y minimizar riesgos.

- **Construcción de portafolios:** *Un agente aprende a reasignar inversiones en acciones y bonos cada semana.*

Objective Portfolio with RL

- **Taller de optimización con RL:** *Uso de simuladores tipo bolsa virtual para entrenar políticas.*

RL for Portfolio Optimization



Recomendadores – Personalización por Refuerzo

AR permite adaptar recomendaciones (películas, productos, noticias) según el comportamiento del usuario, aprendiendo a largo plazo.

- **Tutorial práctico:** *Un agente ajusta qué videos mostrar en una app de streaming según clics, tiempo de visualización y abandono.*

Hands-On RL for Recommenders

- **RL a gran escala:** *Sistemas tipo Netflix o TikTok optimizan su política de sugerencias.*

Scalable RL for Recommenders



Medicina – Tratamientos Personalizados

El aprendizaje por refuerzo puede ayudar a identificar tratamientos óptimos basados en historial y evolución de pacientes.

- **RL en salud:** *Un agente recomienda dosis ajustadas de insulina para pacientes diabéticos.*

Challenges and Promise

- **Identificación de tratamientos:** *Sistemas aprenden qué quimioterapia aplicar según la respuesta al tratamiento.*

Batch RL for Healthcare

RL en Medicina: Sepsis y Diabetes

Optimización del tratamiento de la sepsis en UCIs

- **AI Clinician** (Imperial College London y MIT) utiliza RL para aprender políticas óptimas en UCI. Entrenado con datos de **96,000 pacientes**, logró **reducir la mortalidad** en simulaciones retrospectivas.

Artículo en Nature Medicine

Control automatizado de insulina en diabetes tipo 1

- Algoritmos de AR mejoran sistemas de insulina automatizada para pacientes con **diabetes tipo 1**. Entrenado con datos simulados, logra mantener la **glucosa en rango objetivo** incluso con errores de medición.

Artículo en WIRED



Redes y Comunicaciones – Optimización con RL

AR se usa para asignar dinámicamente canales, potencia y ancho de banda en redes inalámbricas y móviles.

- **Redes celulares:** *Una antena base ajusta en tiempo real la potencia de transmisión para cada usuario. En redes 5G, RL mejora el enrutamiento según la congestión actual.*

Optimizing Wireless Networks with RL



Motivación e intuición

- Inspirado en la psicología conductista: **condicionamiento operante** (Skinner, 1930s).
- Aprendizaje por prueba y error: asociar acciones con recompensas.
- Fundamentos en animales y humanos: aprender del entorno.



Años 1950–1970: Orígenes computacionales

- **1952: Turing** menciona “recompensa” como principio de aprendizaje.
- **1954: Farley y Clark** implementan una red neuronal de refuerzo.
- **1961: Minsky** discute el “credit assignment problem”.
- Algoritmos rudimentarios con poca aplicación real.



1980s: Formalización y agentes aprendices

- **1983: Barto, Sutton y Anderson** desarrollan el algoritmo **Actor-Critic**.
- **1989: Watkins** introduce **Q-learning**, clave en RL moderno.
- Aparición de **TD-learning** y técnicas de programación dinámica.



1990s: Fundamentos teóricos y funciones de valor

- **1992: Sutton y Barto** promueven RL como campo formal.
- **Políticas, funciones de valor, modelos** como pilares del aprendizaje.
- Consolidación del **algoritmo SARSA** y mejoras en exploración/explotación.



2000s: Escalabilidad y aplicaciones reales

- Integración con **control estocástico**, **robótica**, y **economía**.
- Se exploran **políticas aproximadas** y **representaciones continuas**.
- Se desarrolla **RL jerárquico** y aprendizaje **multi-agente**.

2010s: Aprendizaje Profundo por Refuerzo (Deep RL)

- **2013: DeepMind** introduce **DQN** (Deep Q-Network), combinando redes neuronales y Q-learning.
- **AlphaGo (2016)** vence al campeón mundial de Go.
- Grandes avances en **videojuegos, robótica, vehículos autónomos**.

 2020s y futuro: retos y promesas

- Avances en **RL offline**, **exploración segura** y **interpretabilidad**.
- Aplicaciones en **salud**, **finanzas**, **energía**, **sistemas complejos**.
- Desafíos: **generalización**, **eficiencia de datos**, **RL responsable**.



Conclusión

- El aprendizaje por refuerzo ha evolucionado de principios psicológicos a tecnología de punta.
- Su historia refleja el cruce de disciplinas: ciencia cognitiva, teoría de control, IA y más.
- El futuro dependerá de lograr eficiencia, seguridad e impacto social positivo.

Componentes de un Sistema de Aprendizaje por Refuerzo

- **Agente:** el tomador de decisiones.



Ejemplo: Diagrama de interacción
Agente-Ambiente.

Componentes de un Sistema de Aprendizaje por Refuerzo

- **Agente:** el tomador de decisiones.
- **Ambiente:** el entorno con el que interactúa.



Ejemplo: Diagrama de interacción
Agente-Ambiente.

Componentes de un Sistema de Aprendizaje por Refuerzo

- **Agente:** el tomador de decisiones.
- **Ambiente:** el entorno con el que interactúa.
- **Acciones (A):** decisiones que toma el agente.



Ejemplo: Diagrama de interacción
Agente-Ambiente.

Componentes de un Sistema de Aprendizaje por Refuerzo

- **Agente:** el tomador de decisiones.
- **Ambiente:** el entorno con el que interactúa.
- **Acciones (A):** decisiones que toma el agente.
- **Estados (S):** representación del entorno.



Ejemplo: Diagrama de interacción
Agente-Ambiente.

Componentes de un Sistema de Aprendizaje por Refuerzo

- **Agente:** el tomador de decisiones.
- **Ambiente:** el entorno con el que interactúa.
- **Acciones (A):** decisiones que toma el agente.
- **Estados (S):** representación del entorno.
- **Recompensa (R):** señal que guía el aprendizaje.



Ejemplo: Diagrama de interacción
Agente-Ambiente.

Componentes de un Sistema de Aprendizaje por Refuerzo

- **Agente:** el tomador de decisiones.
- **Ambiente:** el entorno con el que interactúa.
- **Acciones (A):** decisiones que toma el agente.
- **Estados (S):** representación del entorno.
- **Recompensa (R):** señal que guía el aprendizaje.
- **Política (π):** estrategia de decisión.



Ejemplo: Diagrama de interacción
Agente-Ambiente.

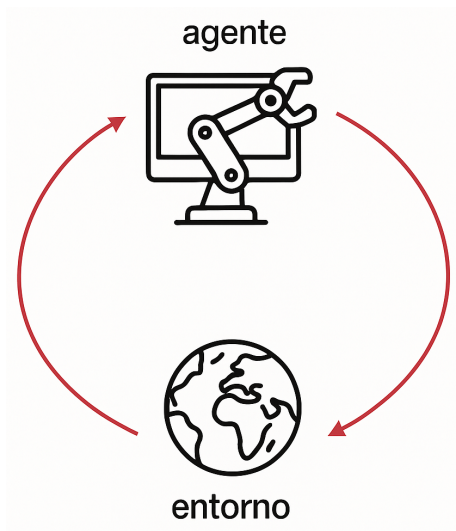
Componentes de un Sistema de Aprendizaje por Refuerzo

- **Agente:** el tomador de decisiones.
- **Ambiente:** el entorno con el que interactúa.
- **Acciones (A):** decisiones que toma el agente.
- **Estados (S):** representación del entorno.
- **Recompensa (R):** señal que guía el aprendizaje.
- **Política (π):** estrategia de decisión.
- **Función de valor (V, Q):** mide la utilidad esperada.



Ejemplo: Diagrama de interacción Agente-Ambiente.

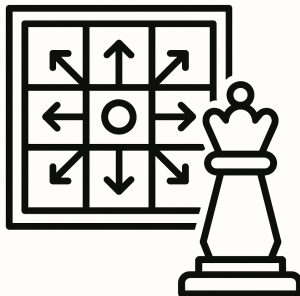
Componentes de un Sistema de Aprendizaje por Refuerzo



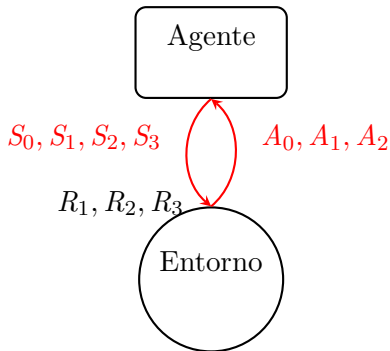
Toma de Decisiones de Manera Secuencial

Meta del Aprendizaje por Refuerzo:

lograr que un agente que aprenda a tomar decisiones secuenciales de manera óptima.

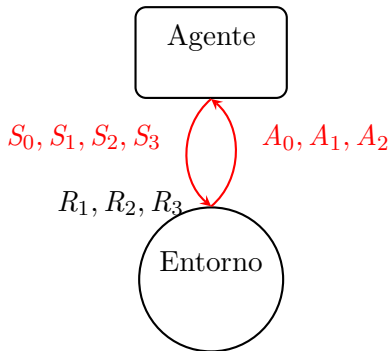


La Interacción Agente-Entorno: Notación Matemática



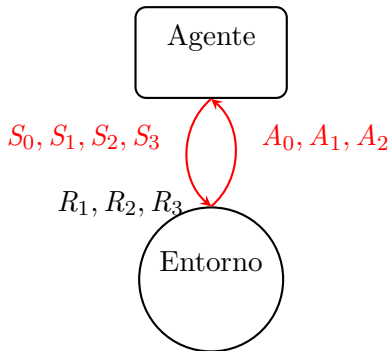
- **Instantes de tiempo:** $t = 0, 1, 2, \dots$

La Interacción Agente-Entorno: Notación Matemática



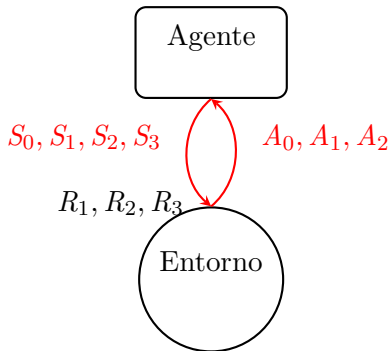
- **Instantes de tiempo:** $t = 0, 1, 2, \dots$
- **Estado en t:** S_t

La Interacción Agente-Entorno: Notación Matemática



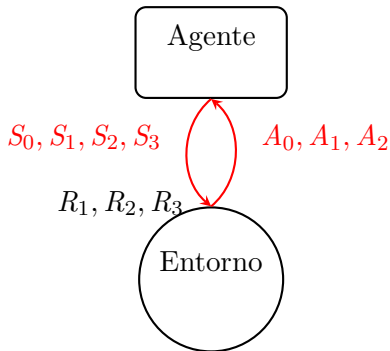
- **Instantes de tiempo:** $t = 0, 1, 2, \dots$
- **Estado en t:** S_t
- **Acción en t:** A_t

La Interacción Agente-Entorno: Notación Matemática



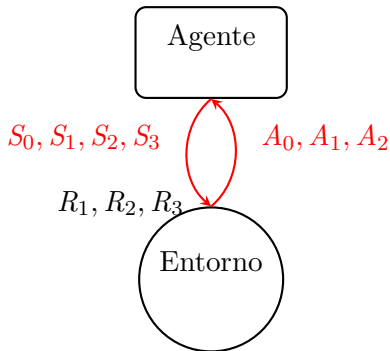
- **Instantes de tiempo:** $t = 0, 1, 2, \dots$
- **Estado en t:** S_t
- **Acción en t:** A_t
- **Recompensa en t:** R_t

La Interacción Agente-Entorno: Notación Matemática



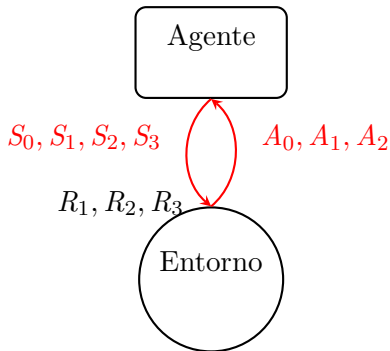
- **Instantes de tiempo:** $t = 0, 1, 2, \dots$
- **Estado en t:** S_t
- **Acción en t:** A_t
- **Recompensa en t:** R_t
- $\mathcal{S} = \{S_0, S_1, \dots, S_m\}$: espacio de estados

La Interacción Agente-Entorno: Notación Matemática



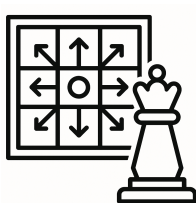
- **Instantes de tiempo:** $t = 0, 1, 2, \dots$
- **Estado en t :** S_t
- **Acción en t :** A_t
- **Recompensa en t :** R_t
- $\mathcal{S} = \{S_0, S_1, \dots, S_m\}$: espacio de estados
- $\mathcal{A} = \{A_0, A_1, \dots, A_n\}$: espacio de acciones

La Interacción Agente-Entorno: Notación Matemática



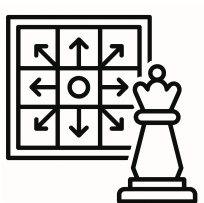
- **Instantes de tiempo:** $t = 0, 1, 2, \dots$
- **Estado en t :** S_t
- **Acción en t :** A_t
- **Recompensa en t :** R_t
- $\mathcal{S} = \{S_0, S_1, \dots, S_m\}$: espacio de estados
- $\mathcal{A} = \{A_0, A_1, \dots, A_n\}$: espacio de acciones
- $\mathcal{R} = \{R_1, R_2, \dots, R_\ell\}$: espacio de recompensas

La Función de Transición: El Componente de Aleatoriedad



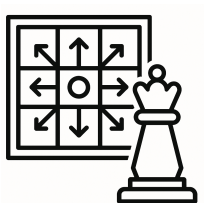
- Torre: 70 % (0.7)

La Función de Transición: El Componente de Aleatoriedad



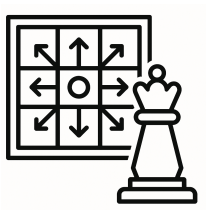
- **Torre:** 70 % (0.7)
- **Caballo:** 25 % (0.25)

La Función de Transición: El Componente de Aleatoriedad



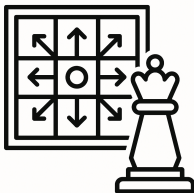
- **Torre:** 70 % (0.7)
- **Caballo:** 25 % (0.25)
- **Peón:** 5 % (0.05)

La Función de Transición: El Componente de Aleatoriedad



- **Torre:** 70 % (0.7)
- **Caballo:** 25 % (0.25)
- **Peón:** 5 % (0.05)

✂ La Función de Transición: El Componente de Aleatoriedad



Función de Transición:

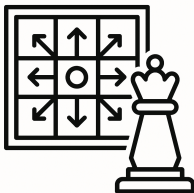
$$p(s' | s, a) = p(S_t = s' | S_{t-1} = s, A_{t-1} = a)$$

- Define la probabilidad de llegar al estado s'

- **Torre:** 70 % (0.7)
- **Caballo:** 25 % (0.25)
- **Peón:** 5 % (0.05)

Este componente modela la aleatoriedad del entorno.

✂ La Función de Transición: El Componente de Aleatoriedad



Función de Transición:

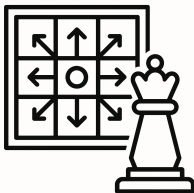
$$p(s' | s, a) = p(S_t = s' | S_{t-1} = s, A_{t-1} = a)$$

- **Torre:** 70 % (0.7)
- **Caballo:** 25 % (0.25)
- **Peón:** 5 % (0.05)

- Define la probabilidad de llegar al estado s'
- Dado que el agente estaba en el estado s y tomó la acción a

Este componente modela la aleatoriedad del entorno.

✂ La Función de Transición: El Componente de Aleatoriedad



Función de Transición:

$$p(s' | s, a) = p(S_t = s' | S_{t-1} = s, A_{t-1} = a)$$

- **Torre:** 70 % (0.7)
- **Caballo:** 25 % (0.25)
- **Peón:** 5 % (0.05)

- Define la probabilidad de llegar al estado s'
- Dado que el agente estaba en el estado s y tomó la acción a
- Es una probabilidad: $p \in (0, 1)$

Este componente modela la aleatoriedad del entorno.



Introducción a los Procesos de Decisión de Markov (MDP)

Un **Proceso de Decisión de Markov (MDP)** es un marco matemático para modelar decisiones secuenciales bajo incertidumbre.



Introducción a los Procesos de Decisión de Markov (MDP)

Un **Proceso de Decisión de Markov (MDP)** es un marco matemático para modelar decisiones secuenciales bajo incertidumbre.

Se define por el conjunto $\langle S, A, R, p \rangle$, donde:

- S : conjunto de estados del entorno.



Introducción a los Procesos de Decisión de Markov (MDP)

Un **Proceso de Decisión de Markov (MDP)** es un marco matemático para modelar decisiones secuenciales bajo incertidumbre.

Se define por el conjunto $\langle S, A, R, p \rangle$, donde:

- S : conjunto de estados del entorno.
- A : conjunto de acciones disponibles.



Introducción a los Procesos de Decisión de Markov (MDP)

Un **Proceso de Decisión de Markov (MDP)** es un marco matemático para modelar decisiones secuenciales bajo incertidumbre.

Se define por el conjunto $\langle S, A, R, p \rangle$, donde:

- S : conjunto de estados del entorno.
- A : conjunto de acciones disponibles.
- $R(s, a)$: recompensa esperada por acción en un estado.



Introducción a los Procesos de Decisión de Markov (MDP)

Un **Proceso de Decisión de Markov (MDP)** es un marco matemático para modelar decisiones secuenciales bajo incertidumbre.

Se define por el conjunto $\langle S, A, R, p \rangle$, donde:

- S : conjunto de estados del entorno.
- A : conjunto de acciones disponibles.
- $R(s, a)$: recompensa esperada por acción en un estado.
- $p(s'|s, a)$: probabilidad de transición de estado.



Introducción a los Procesos de Decisión de Markov (MDP)

Un **Proceso de Decisión de Markov (MDP)** es un marco matemático para modelar decisiones secuenciales bajo incertidumbre.

Se define por el conjunto $\langle S, A, R, p \rangle$, donde:

- S : conjunto de estados del entorno.
- A : conjunto de acciones disponibles.
- $R(s, a)$: recompensa esperada por acción en un estado.
- $p(s'|s, a)$: probabilidad de transición de estado.



Introducción a los Procesos de Decisión de Markov (MDP)

Un **Proceso de Decisión de Markov (MDP)** es un marco matemático para modelar decisiones secuenciales bajo incertidumbre.

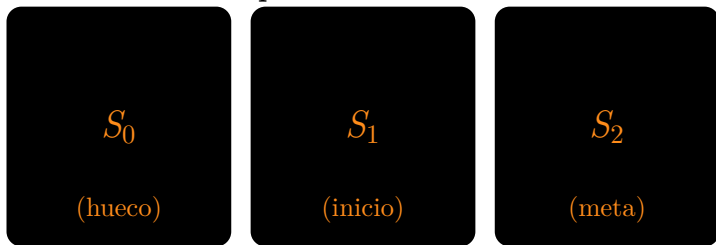
Se define por el conjunto $\langle S, A, R, p \rangle$, donde:

- S : conjunto de estados del entorno.
- A : conjunto de acciones disponibles.
- $R(s, a)$: recompensa esperada por acción en un estado.
- $p(s'|s, a)$: probabilidad de transición de estado.

Satisface la **propiedad de Markov**: el siguiente estado depende solo del estado actual y la acción tomada, no del historial completo

Ejemplo: tablero unidimensional

Espacio de Estados



$$S = \{S_0, S_1, S_2\}$$

$$A = \{A_0, A_1\}$$

$$R = \{+1, 0\}$$

donde A_0 : Izquierda, A_1 : Derecha.

Función de transición

$S_{t-1}(s)$	$A_{t-1}(a)$	$S_t(s')$	$p(s' \mid s, a)$	R_t

Función de transición

$S_{t-1}(s)$	$A_{t-1}(a)$	$S_t(s')$	$p(s' \mid s, a)$	R_t
S_0	A_0	S_0	1	0

Función de transición

$S_{t-1}(s)$	$A_{t-1}(a)$	$S_t(s')$	$p(s' \mid s, a)$	R_t
S_0	A_0	S_0	1	0
S_0	A_1	S_0	1	0

Función de transición

$S_{t-1}(s)$	$A_{t-1}(a)$	$S_t(s')$	$p(s' \mid s, a)$	R_t
S_0	A_0	S_0	1	0
S_0	A_1	S_0	1	0
S_1	A_0	S_0	1	0

Función de transición

$S_{t-1}(s)$	$A_{t-1}(a)$	$S_t(s')$	$p(s' \mid s, a)$	R_t
S_0	A_0	S_0	1	0
S_0	A_1	S_0	1	0
S_1	A_0	S_0	1	0
S_1	A_1	S_2	1	+1

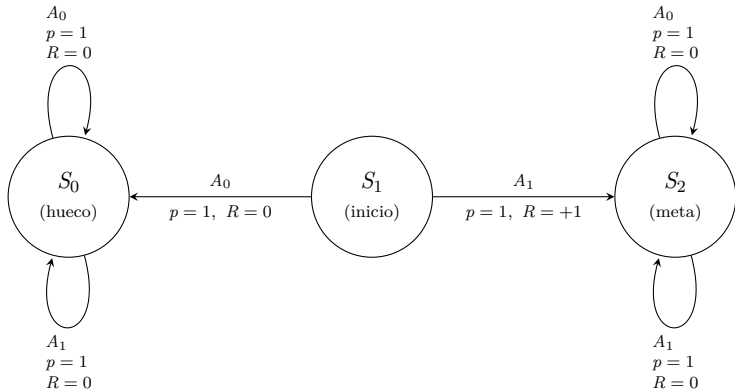
Función de transición

$S_{t-1}(s)$	$A_{t-1}(a)$	$S_t(s')$	$p(s' \mid s, a)$	R_t
S_0	A_0	S_0	1	0
S_0	A_1	S_0	1	0
S_1	A_0	S_0	1	0
S_1	A_1	S_2	1	+1
S_2	A_0	S_2	1	0

Función de transición

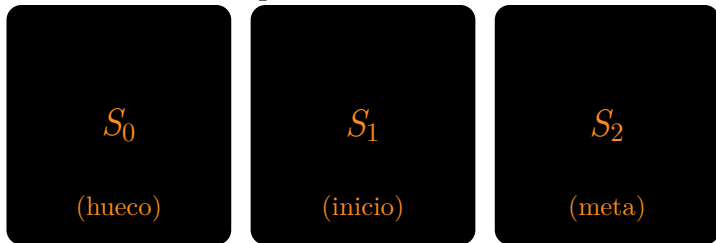
$S_{t-1}(s)$	$A_{t-1}(a)$	$S_t(s')$	$p(s' \mid s, a)$	R_t
S_0	A_0	S_0	1	0
S_0	A_1	S_0	1	0
S_1	A_0	S_0	1	0
S_1	A_1	S_2	1	+1
S_2	A_0	S_2	1	0
S_2	A_1	S_2	1	0

Grafo de la Función de transición



Ejemplo: tablero unidimensional con componente estocástico

Espacio de Estados



$$S = \{S_0, S_1, S_2\}$$

$$A = \{A_0, A_1\}$$

$$R = \{+1, 0\}$$

$$A_0 : \begin{cases} \text{izquierda} : 80 \% \\ \text{derecha} : 20 \% \end{cases}$$

$$A_1 : \begin{cases} \text{derecha} : 80 \% \\ \text{izquierda} : 20 \% \end{cases}$$

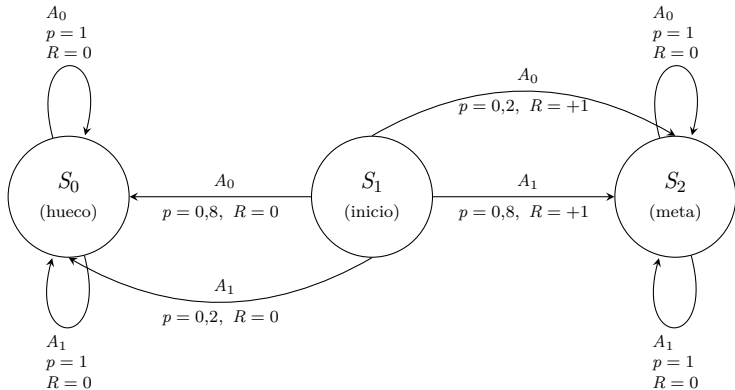
Función de transición estocástica

$S_{t-1}(s)$	$A_{t-1}(a)$	$S_t(s')$	$p(s' \mid s, a)$	R_t
S_0	A_0	S_0	1	0

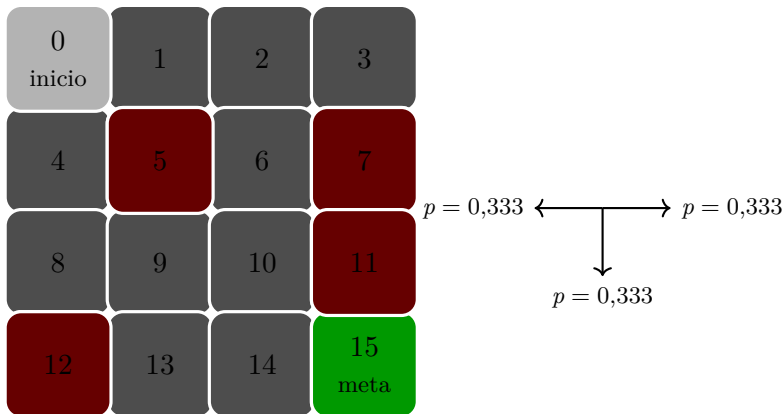
Función de transición estocástica

$S_{t-1}(s)$	$A_{t-1}(a)$	$S_t(s')$	$p(s' \mid s, a)$	R_t
S_0	A_0	S_0	1	0
S_0	A_1	S_0	1	0
S_1	A_0	S_0	0.8	0
S_1	A_0	S_2	0.2	+1
S_1	A_1	S_2	0.8	+1
S_1	A_1	S_0	0.2	0
S_2	A_0	S_2	1	0
S_2	A_1	S_2	1	0

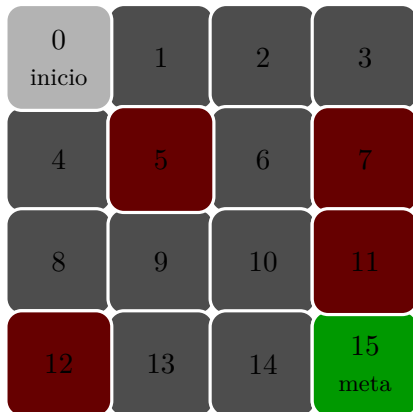
Transiciones estocásticas



Ejemplo: tablero bidimensional



Espacio de Estados



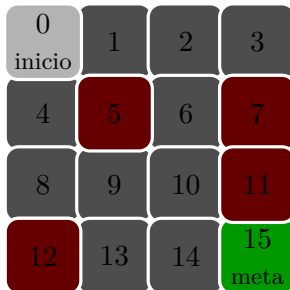
$$S = \{S_0, S_1, S_2, S_3, S_4, \dots, S_{15}\}$$

Espacio de estados: discreto o continuo, finito o infinito o indeterminado.

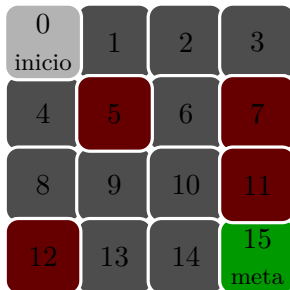


La propiedad de Markov: notación matemática

$$\underbrace{P(S_{t+1} \mid S_t, A_t)}_{\text{estado futuro} \mid \text{estado-acción presente}} = \underbrace{P(S_{t+1} \mid S_t, A_t, S_{t-1}, A_{t-1}, S_{t-2}, A_{t-2}, \dots)}_{\text{estado futuro} \mid \text{historial de pares estado-acción}}$$

Transiciones desde S_0 

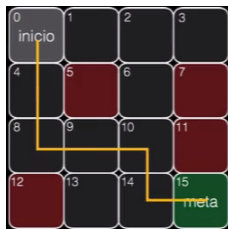
S	A	S'	p	R
S_0	$A_0 \leftarrow$	S_0	0.33	0
S_0	$A_0 \uparrow$	S_0	0.33	0
S_0	$A_0 \downarrow$	S_4	0.33	0
S_0	$A_1 \downarrow$	S_4	0.33	0
S_0	$A_1 \leftarrow$	S_0	0.33	0
S_0	$A_1 \rightarrow$	S_1	0.33	0
S_0	$A_2 \rightarrow$	S_1	0.33	0
S_0	$A_2 \uparrow$	S_0	0.33	0
S_0	$A_2 \downarrow$	S_4	0.33	0
S_0	$A_3 \uparrow$	S_0	0.33	0
S_0	$A_3 \leftarrow$	S_0	0.33	0
S_0	$A_3 \rightarrow$	S_1	0.33	0

Transiciones desde S_{14} 

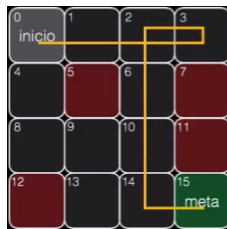
S	A	S'	p	R
S_{14}	$A_0 \leftarrow$	S_{13}	0.33	0
S_{14}	$A_0 \uparrow$	S_{10}	0.33	0
S_{14}	$A_0 \downarrow$	S_{14}	0.33	0
S_{14}	$A_1 \downarrow$	S_{14}	0.33	0
S_{14}	$A_1 \leftarrow$	S_{13}	0.33	0
S_{14}	$A_1 \rightarrow$	S_{15}	0.33	+1
S_{14}	$A_2 \rightarrow$	S_{15}	0.33	+1
S_{14}	$A_2 \uparrow$	S_{10}	0.33	0
S_{14}	$A_2 \downarrow$	S_{14}	0.33	0
S_{14}	$A_3 \uparrow$	S_{10}	0.33	0
S_{14}	$A_3 \leftarrow$	S_{13}	0.33	0
S_{14}	$A_3 \rightarrow$	S_{15}	0.33	+1

Horizonte

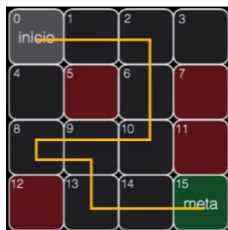
Horizonte: la duración de la interacción del agente con el entorno



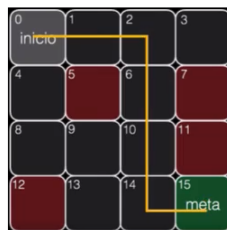
Horizonte = 6



Horizonte = 8

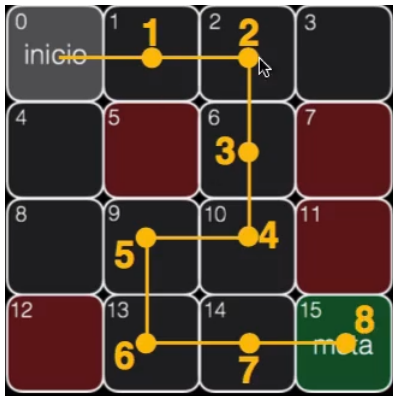


Horizonte = 10



Horizonte = 6

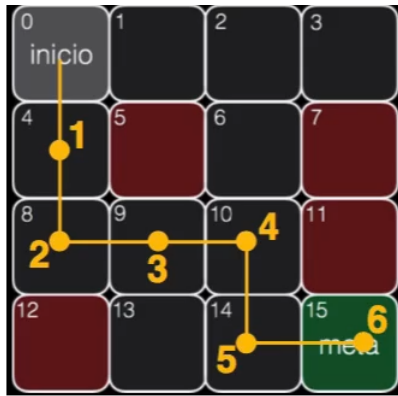
Dos rutas y recompensa total



Ruta1

$$Recompensa = 0 + 0 + 0 + 0$$

$$+ 0 + 0 + 0 + 1 = \boxed{1}$$

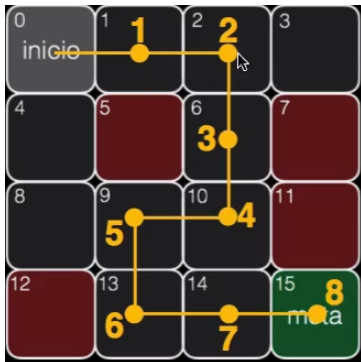


Ruta2

$$Recompensa = 0 + 0 + 0 + 0$$

$$+ 0 + 1 = \boxed{1}$$

Retorno (G): suma recompensas individuales



Ruta1

$$\begin{aligned}
 G_0 &= R_1 + R_2 + R_3 + R_4 \\
 &\quad + R_5 + R_6 + R_7 + R_8 \\
 G_0 &= 0 + 0 + 0 + 0 + 0 + 0 + 0 + 1
 \end{aligned}$$

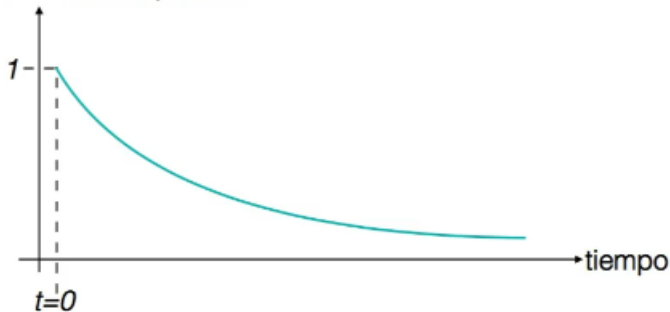


Ruta2

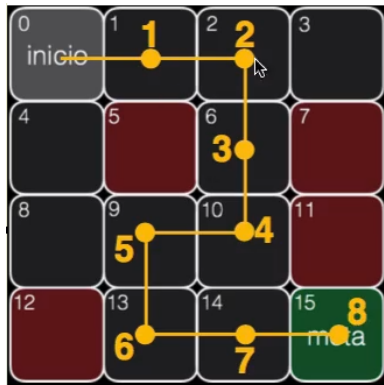
$$\begin{aligned}
 G_0 &= R_1 + R_2 + R_3 + R_4 \\
 &\quad + R_5 + R_6 \\
 G_0 &= 0 + 0 + 0 + 0 + 0 + 1
 \end{aligned}$$

El descuento: importancia recompensas en el tiempo

Ponderación de las recompensas



Retorno con descuento en la ruta 1



$\gamma = 0,99$ (factor de descuento)

$$\begin{aligned}
 G_0 &= (\gamma^0) 0 + (\gamma^1) 0 + (\gamma^2) 0 + \\
 &\quad (\gamma^3) 0 + (\gamma^4) 0 + (\gamma^5) 0 + \\
 &\quad (\gamma^6) 0 + (\gamma^7) 1 \\
 &= \gamma^7
 \end{aligned}$$

$$G_0 = 0,99^7 \approx \boxed{0,9321}$$

$$\begin{aligned}
 G_0 &= R_1 + R_2 + R_3 + R_4 + R_5 + R_6 + R_7 + R_8 \\
 &= 0 + 0 + 0 + 0 + 0 + 0 + 0 + 1 = \boxed{1}
 \end{aligned}$$

Retorno con descuento en la ruta 2



$\gamma = 0,99$ (factor de descuento)

$$G_0 = (\gamma^0) 0 + (\gamma^1) 0 + (\gamma^2) 0 + (\gamma^3) 0 + (\gamma^4) 0 + (\gamma^5)$$

$$= \gamma^5$$

$$G_0 = 0,99^5 \approx \boxed{0,950}$$

$$G_0 = R_1 + R_2 + R_3 + R_4 + R_5 + R_6$$

$$= 0 + 0 + 0 + 0 + 0 + 1 = \boxed{1}$$

Retorno

- **Retorno sin descuento:**

$$G_t = R_{t+1} + R_{t+2} + R_{t+3} + \dots$$

- **Retorno con descuento:** $\gamma \in (0, 1]$

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \dots$$

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

(forma recursiva: $G_t = R_{t+1} + \gamma G_{t+1}$)



Procesos de Decisión de Markov (MDP)

Un **Proceso de Decisión de Markov (MDP)** es un marco matemático para modelar decisiones secuenciales bajo incertidumbre.

 Procesos de Decisión de Markov (MDP)

Un **Proceso de Decisión de Markov (MDP)** es un marco matemático para modelar decisiones secuenciales bajo incertidumbre.

Se define por el conjunto $\langle S, A, p, R, H, \gamma \rangle$, donde:

- S : espacio de estados.



Procesos de Decisión de Markov (MDP)

Un **Proceso de Decisión de Markov (MDP)** es un marco matemático para modelar decisiones secuenciales bajo incertidumbre.

Se define por el conjunto $\langle S, A, p, R, H, \gamma \rangle$, donde:

- S : espacio de estados.
- A : espacio de acciones.



Procesos de Decisión de Markov (MDP)

Un **Proceso de Decisión de Markov (MDP)** es un marco matemático para modelar decisiones secuenciales bajo incertidumbre.

Se define por el conjunto $\langle S, A, p, R, H, \gamma \rangle$, donde:

- S : espacio de estados.
- A : espacio de acciones.
- $p(s'|s, a)$: función de transición de estado (reglas de juego).



Procesos de Decisión de Markov (MDP)

Un **Proceso de Decisión de Markov (MDP)** es un marco matemático para modelar decisiones secuenciales bajo incertidumbre.

Se define por el conjunto $\langle S, A, p, R, H, \gamma \rangle$, donde:

- S : espacio de estados.
- A : espacio de acciones.
- $p(s'|s, a)$: función de transición de estado (reglas de juego).
- $R(s, a)$: recompensa (premio o castigo).



Procesos de Decisión de Markov (MDP)

Un **Proceso de Decisión de Markov (MDP)** es un marco matemático para modelar decisiones secuenciales bajo incertidumbre.

Se define por el conjunto $\langle S, A, p, R, H, \gamma \rangle$, donde:

- S : espacio de estados.
- A : espacio de acciones.
- $p(s'|s, a)$: función de transición de estado (reglas de juego).
- $R(s, a)$: recompensa (premio o castigo).
- H : horizonte (duración en el tiempo interacción agente-entorno).



Procesos de Decisión de Markov (MDP)

Un **Proceso de Decisión de Markov (MDP)** es un marco matemático para modelar decisiones secuenciales bajo incertidumbre.

Se define por el conjunto $\langle S, A, p, R, H, \gamma \rangle$, donde:

- S : espacio de estados.
- A : espacio de acciones.
- $p(s'|s, a)$: función de transición de estado (reglas de juego).
- $R(s, a)$: recompensa (premio o castigo).
- H : horizonte (duración en el tiempo interacción agente-entorno).
- $\gamma \in [0, 1]$: factor de descuento futuro.



Procesos de Decisión de Markov (MDP)

Un **Proceso de Decisión de Markov (MDP)** es un marco matemático para modelar decisiones secuenciales bajo incertidumbre.

Se define por el conjunto $\langle S, A, p, R, H, \gamma \rangle$, donde:

- S : espacio de estados.
- A : espacio de acciones.
- $p(s'|s, a)$: función de transición de estado (reglas de juego).
- $R(s, a)$: recompensa (premio o castigo).
- H : horizonte (duración en el tiempo interacción agente-entorno).
- $\gamma \in [0, 1]$: factor de descuento futuro.



Procesos de Decisión de Markov (MDP)

Un **Proceso de Decisión de Markov (MDP)** es un marco matemático para modelar decisiones secuenciales bajo incertidumbre.

Se define por el conjunto $\langle S, A, p, R, H, \gamma \rangle$, donde:

- S : espacio de estados.
- A : espacio de acciones.
- $p(s'|s, a)$: función de transición de estado (reglas de juego).
- $R(s, a)$: recompensa (premio o castigo).
- H : horizonte (duración en el tiempo interacción agente-entorno).
- $\gamma \in [0, 1]$: factor de descuento futuro.

Satisface la **propiedad de Markov**: el siguiente estado depende solo del estado actual y la acción tomada, no del historial completo



El objetivo del agente

El agente toma decisiones (acciones) en un entorno, recibiendo observaciones y recompensas como retroalimentación.

🎯 El objetivo del agente

El agente toma decisiones (acciones) en un entorno, recibiendo observaciones y recompensas como retroalimentación.

★ Objetivo principal

Encontrar una secuencia de acciones (aprender una **política** π) que maximice la recompensa total esperada (retorno acumulado).

El objetivo del agente

El agente toma decisiones (acciones) en un entorno, recibiendo observaciones y recompensas como retroalimentación.

Objetivo principal

Encontrar una secuencia de acciones (aprender una **política** π) que maximice la recompensa total esperada (retorno acumulado).

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

 r_{t+k+1} : recompensa en el tiempo $t + k + 1$

 $\gamma \in [0, 1]$: factor de descuento para recompensas futuras

_____ **Meta:** Maximizar $\mathbb{E}[G_t]$, la expectativa del retorno desde cada estado.

El reto del objetivo del agente

EL RETO: LA TOMA DE DECISIONES DE MANERA ÓPTIMA



$$G_0 = (0.99)^0 * 0 + (0.99)^1 * 0 + (0.99)^2 * 0 + (0.99)^3 * 0 + (0.99)^4 * 0 + (0.99)^5 * 1 = 0.950$$

El reto del objetivo del agente

EL RETO: LA TOMA DE DECISIONES DE MANERA ÓPTIMA

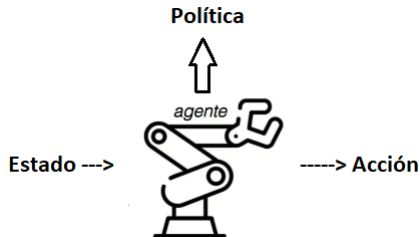


$$G_0 = (0.99)^0 * 0 + (0.99)^1 * 0 + (0.99)^2 * 0 + (0.99)^3 * 0 + (0.99)^4 * 0 + (0.99)^5 * 1 = 0.950$$





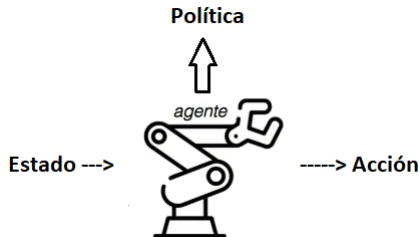
La política



Una política π define el comportamiento del agente: qué acción tomar en cada estado.



La política



Una política π define el comportamiento del agente: qué acción tomar en cada estado.

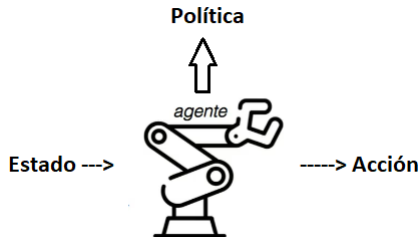


Tipos de políticas

➔ **Determinista:** $\pi(s) = a$, asocia una acción específica al estado s .



La política



Una política π define el comportamiento del agente: qué acción tomar en cada estado.



Tipos de políticas

- ➔ **Determinista:** $\pi(s) = a$, asocia una acción específica al estado s .
- ✂ **Estocástica:** $\pi(a|s)$, asigna una probabilidad a cada acción posible en el estado s .



La política

$$\underbrace{\pi(a \mid s)}_{\text{política}} = \underbrace{\mathbb{P}(A_t = a \mid S_t = s)}_{\text{probabilidad (0-1)}}$$



La política

Para el estado s_{10} :

$$\underbrace{\pi(a \mid s)}_{\text{política}} = \underbrace{\mathbb{P}(A_t = a \mid S_t = s)}_{\text{probabilidad (0-1)}}$$

$$\pi(A_0 \mid s_{10}) = 0,10 \quad (\leftarrow)$$

$$\pi(A_1 \mid s_{10}) = 0,75 \quad (\downarrow)$$

$$\pi(A_2 \mid s_{10}) = 0,05 \quad (\rightarrow)$$

$$\pi(A_3 \mid s_{10}) = 0,08 \quad (\uparrow)$$

Objetivo: encontrar la Política óptima π a medida que este interactúa con su entorno.



Elección de acciones

La política guía al agente en la toma de decisiones, influida por el valor esperado de los estados o acciones. (usa únicamente el estado para predecir la acción a ejecutar)



Función estado-valor $V(s)$

¿Cómo comparar diferentes políticas?


 π_1

 π_2

Qué hace

Función estado-valor permiten cuantificar la bondad de un estado en particular. Pone números a los diferentes estados alcanzados por el Agente y permite comparar diferentes políticas en un entorno estocástico.

El Retorno Esperado

Proceso estocástico
+
política



1) No hay secuencias
de acciones arbitrarias



2) Estocástico $\Rightarrow$
múltiples secuencias de acciones

Retorno Esperado

Es el resultado de promediar todos los posibles retornos que se obtendrán tras analizar todas las posibles secuencias de acciones.

El valor de un estado

Definición: El **valor de un estado** es el **retorno esperado** que se obtendría si el agente partiera del estado s y ejecutara la secuencia de acciones definida por la política π .



π estocástica con $p = 0,33$ por acción válida. Para s_{14} :

- $s_{14} \rightarrow s_{15} : 33\%$ valor = 10
- $s_{14} \rightarrow s_{10} : 33\%$ valor = 9
- $s_{14} \rightarrow s_{13} : 33\%$ valor = 2

$$v_{\pi}(s_{14}) = \mathbb{E}_{\pi}[G_t \mid S_t = s_{14}] = \frac{10 + 9 + 2}{3}$$



Función estado-valor $v(s)$

La función estado-valor $v_\pi(s)$ asigna a cada estado s el valor esperado del retorno cuando se inicia en s y se sigue la política π . Sirve para evaluar la eficacia de π estado por estado.

$$v_\pi(s) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s \right]$$



Interpretación

Mide cuán beneficioso es estar en el estado s bajo la política π .

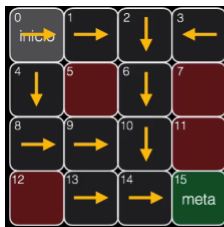
Aplicaciones:

- ✓ Evaluar estados
- ⬆ Mejorar políticas
- ⚙ Base para algoritmos como TD y programación dinámica



Función acción-valor

¿Cuál acción es mejor?


 π_1

 π_2

Qué hace

Función acción-valor permite determinar el valor de tomar una acción partiendo de un estado en particular: cuál acción es mejor en un estado específico.

Q Función acción-valor $Q(s, a)$

Definición: La **función acción-valor** (o función “Q”) permite calcular el **retorno esperado** que obtendría el agente al tomar la acción a estando en el estado s y siguiendo la política π .

$$Q_{\pi}(s, a) = \mathbb{E}_{\pi} \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s, A_t = a \right]$$

Interpretación

Mide la calidad de tomar la acción a desde el estado s bajo la política π .

Usos clave:

-  Comparar acciones posibles en un estado
-  Derivar políticas óptimas: $\pi^*(s) = \arg \max_a Q^*(s, a)$
-  Base de algoritmos como Q-learning y SARSA

↕ Ecuaciones para las funciones estado-valor y acción-valor

Forma alternativa de representar matemáticamente estas funciones y que servirán como punto de partida para los algoritmos clásicos de Aprendizaje por Refuerzo

➡ Para $v_{\pi}(s)$:

$$v_{\pi}(s) = \mathbb{E}_{\pi} [G_t | S_t = s]$$

➡ Para $Q_{\pi}(s, a)$:

$$Q_{\pi}(s, a) = \mathbb{E}_{\pi} [G_t | S_t = s, A_t = a]$$

↺↻ Ecuación de Bellman para función estado-valor

Relaciones recursivas que expresan funciones de valor en términos de recompensas inmediatas y valores futuros.

➡ Para $v_{\pi}(s)$:

$$v_{\pi}(s) = \mathbb{E}_{\pi} [G_t | S_t = s]$$

$$\Downarrow$$

$$v_{\pi}(s) = \sum_a \pi(a|s) \sum_{s', r} p(s', r|s, a) [r + \gamma v_{\pi}(s')]$$



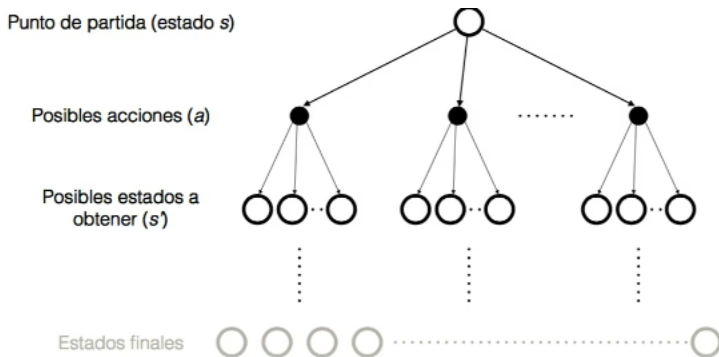
¿Por qué son importantes?



Richard Bellman
(1920-1984)

Permiten evaluar y mejorar políticas, y son la base de muchos algoritmos de RL.

Interpretación: Ecuación de Bellman para función estado-valor



↺↻ Ecuación de Bellman para función acción-valor

Relaciones recursivas que expresan funciones de valor en términos de recompensas inmediatas y valores futuros.

➡ Para $Q_\pi(s, a)$:

$$Q_\pi(s, a) = \mathbb{E}_\pi [G_t \mid S_t = s, A_t = a]$$

⇓

$$Q_\pi(s, a) = \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_\pi(s')]$$



¿Por qué son importantes?

Permiten evaluar y mejorar políticas, y son la base de muchos algoritmos de RL.



Richard Bellman
(1920-1984)

Función acción-ventaja (advantage)

Definición

$$a_{\pi}(s, a) = Q_{\pi}(s, a) - v_{\pi}(s)$$

Mide cuánto *mejor o peor* es elegir a en s respecto al valor promedio del estado bajo π .

Función acción-ventaja (advantage)

Intuición y uso

- Centra el valor de acción restando la base $v_\pi(s) \Rightarrow$ estabiliza y reduce varianza.

Ejemplo (para un estado s)

$$v_\pi(s) = 7,2, \quad Q_\pi(s, a_1) = 8,0, \quad Q_\pi(s, a_2) = 6,5$$

$$a_\pi(s, a_1) = 8,0 - 7,2 = 0,8 \quad (\text{mejor}) \quad a_\pi(s, a_2) = 6,5 - 7,2 = -0,7 \quad (\text{peor})$$



¿Con o sin modelos?

$$\{S, A, R, p, \gamma\}$$

basados en modelos
(*model-based*)



libres de modelos
(*model-free*)





Clasificación algoritmos RL

	PREDICCIÓN	CONTROL
BASADOS EN MODELOS (PLANEACIÓN)	- Programación dinámica: evaluación de la política	- Programación dinámica: iteración de la política, iteración de valores
LIBRES DE MODELOS (APRENDIZAJE)	- Predicción con Monte Carlo - Aprendizaje por Diferencias Temporales	- Control con Monte Carlo - SARSA - Q-learning